SLIDE 1

What the agents actually said to each other

Every figure on this page is emitted by one script over the raw logs: replication_messages.py

THE SETUP — what the two agents are being asked to do

The task	Two AI agents are each given **one feature to add to the same codebase** — for example, two new options on the same function.
The catch	Each works in its **own private copy**. Neither can see the other's edits. They **can send each other messages** while they work.
The join	At the end, their two sets of edits are combined **automatically**, with no human to resolve disagreements.
What can fail	If both rewrote the **same lines**, the automatic combine gives up — a **clash**. The work is done, but it cannot be assembled.

How the messages were turned into numbers

- We collected **every message** the agents sent — **703 across 147 attempts** — and linked each conversation to what happened to that attempt, then compared the talk against the **actual edits** both agents produced.
- One script does all of it: `replication_messages.py`. It reads the raw logs, prints every figure below, and can be rerun by anyone. **Nothing here is hand-counted and nothing is estimated.**

① What can be counted without interpretation

Measure	Value	What the machine literally looks for
Attempts in which they name at least one **file**	**100%**	a filename token (*something*.py, .go, .rs …) appears
Attempts in which either agent asks a **question**	**9%**	a "?" appears anywhere in the conversation
Attempts in which they exchange **actual code**	**3%**	a code block or a function definition appears — a lower bound
Messages sent per attempt (median)	**5**	count of messages in the run's log
Length of the whole conversation (median)	**114s**	last message timestamp minus first

② What the two sets of edits show — the load-bearing evidence

On the attempts that failed to combine	Value	How it is computed
Conflicting attempts examined	**93**	runs whose combine failed, with both diffs present
…the two diffs touch a **shared file**	**100%**	filenames parsed from both diffs, intersected
…that file was **named in the chat**	**100%**	colliding filename matched against the conversation text
…they collide on the **same starting line**	**90%**	diff hunk headers (@@ -start,len @@) compared between the two diffs
…both rewrote the **same function definition**	**27%**	"def name(" lines extracted from both diffs, intersected

These come from parsing the diffs, so they do not depend on reading the agents' language at all.

The pattern in one attempt (quoted verbatim; this attempt failed to combine)

agent 2 — *"Suggest final combined order:* `(environment, value, attribute, default=None, separator='.', reverse=False)` *— just append your reverse param after mine."*

agent 1 — *"my working copy doesn't show your separator changes… we appear to be in separate sandboxes. Since each of us submits our own work independently, this should be fine — no actual clobbering."*

Agent 2 spells out the correct combined line; agent 1 agrees; **neither types it**. One documented example, shown to illustrate the mechanism — not offered as a frequency.

They agree on a description of the answer. But the combine is decided by the exact characters they type.

What we deliberately do not claim. An earlier pass scored the conversations for habits like "both said they'd add theirs last" or "declared it would combine fine". Those depend on a hand-written list of phrases, are lower bounds by construction, and were never checked by a second coder — so they are excluded here rather than quoted as measurements. Every figure above is either a literal count or derived from the diffs.

SLIDE 2

Six sets of rules, and what each one tries to fix

Each rung forces the pair to agree on more of the final code than the rung below

- We re-ran the same task six times over, changing **only the rules the agents are given for working together**. Same AI model, same tasks, same marking, same automatic combine — so any difference is caused by the **rules**, nothing else.
- The six form a ladder: from agreeing on **nothing** at the bottom, to agreeing on **the exact code** at the top.

	Rule set	They must agree on…	What it is trying to fix	Combined / both work
1	**control**	Nothing — no messaging at all	The baseline. If the talking is doing real work, removing it should hurt.	13% / 2%
2	**free-text**	Whatever they choose to say	The original condition, repeated here so the other five have something to beat.	21% / 3%
3	**semi-structured**	A filled-in form about their intent	Tests whether the problem was just **sloppiness**: forces every message to name its files and intent, or be rejected.	16% / 3%
4	**plan-handshake**	A split of the files, agreed up front	Fixes the missing hand-out of work: they must **agree who owns which file and wait for a reply** before typing anything.	20% / 10%
5	**designated-coder**	One owner per shared file	Accepts that a file both need cannot be split, so **only one agent writes it**; the other sends a written request instead.	18% / 58%
6	**coauthor-overlap**	**The exact merged code itself**	Goes after the clash directly: they agree the shared code word-for-word, then **both type the identical text**.	**78% / 69%**

Why these six, and why in this order

- Slide 1 showed the clashes happen **inside** a file the agents had already talked about, on the **same line**. Talking about *files* is therefore too coarse to prevent them.
- That lets us **predict the result before running anything**: rules 1–4 only ever get the pair to agree at the level of files or intentions, so they should fail. Only rules 5–6 reach the actual lines of code, so only they should work.
- Rules 3 and 4 are the useful failures. **3** shows the agents were not being vague — they already named their files every single time. **4** shows the problem is not poor division of labour — forcing a real agreed split made the code more *correct*, but no easier to combine.

> **TAKEAWAY** *Each rung takes away one more thing the two agents can disagree about. The top rung takes away all of them.*

Result columns are the same figures as slide 3, produced by `analyze.py` over the run logs and frozen in `data/nano_study.json`.

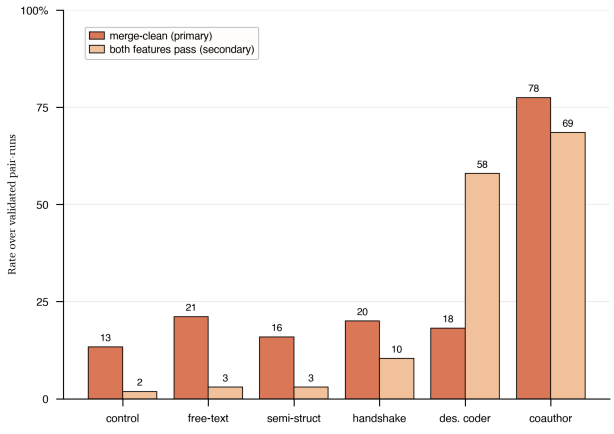
SLIDE 3

What happened

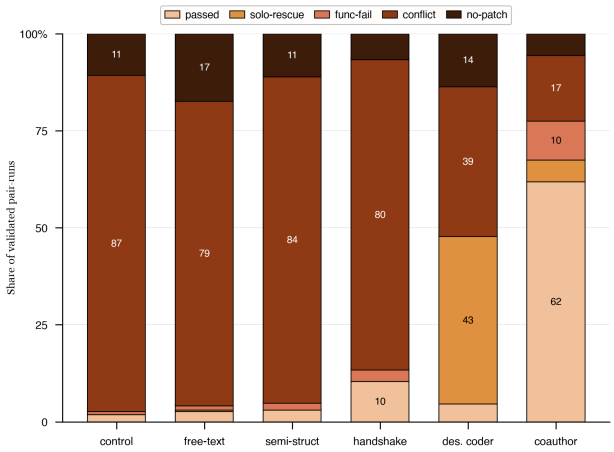
18 tasks chosen because the two features genuinely overlap · ~1,250 attempts · Claude Sonnet 5

READING THE NUMBERS — the two things being measured

Combined cleanly	The two sets of edits went together automatically, with no clash. *The headline measure.*
Both features work	After combining, both agents' features actually pass their tests. *The stricter measure.*
Clash	Both agents rewrote the same lines, so the automatic combine refused.
One carried the pair	Both features work, but only because **one** agent's copy happened to contain both — not because the two combined.



How often each rule set worked. Taller is better. Five of the six sit in the same low band — only the last one (coauthor-overlap) breaks out.



How the attempts failed. Each attempt in exactly one bucket. The dark band is clashes: it dominates everywhere except the last rule set.

Rule set	Combined cleanly	Both features work	Clashed
control	13%	2%	87%
free-text	21%	3%	79%
semi-structured	16%	3%	84%
plan-handshake	20%	10%	80%
designated-coder	18%	58%	39%
coauthor-overlap	**78%**	**69%**	**17%**

Key findings

- **The rules matter enormously — but only one set of rules worked.** Making the pair co-write the shared code took clean combining from **13% to 78%**, and both-features-working from **2% to 69%**. It is the only one that beats the baseline once you account for having tried six things.
- **Talking more achieved nothing.** Tidying up the messages, or planning who owns which file, left the result no better than **giving them no way to talk at all** — exactly as slide 1 predicted.
- **It works by changing what gets typed, not how much gets said.** The winning rule set produced **26** cases where both agents wrote byte-for-byte identical code; the other five produced **1** between them across more than a thousand attempts.
- **Working code and combinable code are different problems.** designated-coder gets both features working 58% of the time while still clashing as often as the baseline — one agent quietly carried the pair.
- **Still not solved.** Even the best rule set clashes 17% of the time, and another 10% combine cleanly but are broken. One AI model, one setup, one style of automatic combine.

TAKEAWAY *Rules for cooperation only help when they constrain **what the agents type** — not how much they say to each other.*

Source. Every number and both figures on this page come from `analyze.py`, which reads the per-run evaluation records under `logs/` and writes `data/nano_study.json`; the figures are drawn from that JSON by `figures.py`. "Beats the baseline" means significant under a Cochran–Mantel–Haenszel test stratified by task, Holm-corrected across the eight comparisons made.